

How Uber Finds Nearby Drivers at 1 Million Requests per Second

#31: And How Proximity Service Works Explained Like You're Twelve (5 minutes)



NEO KIM

JAN 04, 2024  PAID

Get my system design playbook for FREE on newsletter signup:

✓ Subscribed

This post outlines how Uber finds nearby drivers at 1 million requests per second. If you want to learn more, scroll to the bottom and find the references.

- *Share this post & I'll send you some rewards for the referrals.*

March 2016 - Istanbul, Turkey.

Charles and his family are on a tourist visit.

Yet it was difficult for them to travel around because the nearest taxi stand was 3 km away from the hotel.

So they were sad.



newsletter.systemdesign.one

They hear about the ride-sharing app called Uber from the hotel receptionist.

They installed it immediately and were stunned by the easiness of finding a driver.

How Does Uber Find Nearby Drivers?

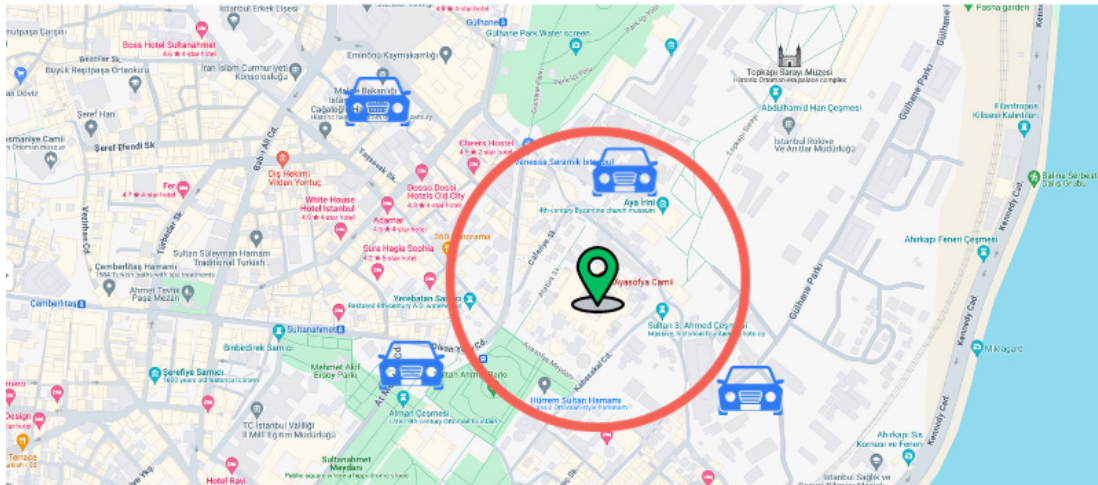
Finding nearby drivers with accuracy and scalability is a hard problem.

Here's how Uber solves it:

1. Location Indexing

It's difficult to find nearby drivers only based on latitude and longitude.

So they index locations to find nearby drivers efficiently.



newsletter.systemdesign.one

Finding Nearby Drivers

They use the *H3* library for location indexing.

H3 is a hexagonal-shaped hierarchical geospatial indexing system created at Uber.

H3 divides Earth's surface into cells on a flat grid, giving each cell a unique identifier with a 64-bit integer.

An H3 index is a reference to a specific cell on the grid.



newsletter.systemdesign.one

An H3 Index Referencing a Specific Cell

They find nearby drivers by identifying the relevant cells covering the rider's area. And then listing the drivers in those cells sorted by the estimated time of arrival (ETA).

H3 offers the benefits of a *hierarchical indexing system* and a *hexagonal grid system*.

Here's how H3 works:

i) Hierarchical Index

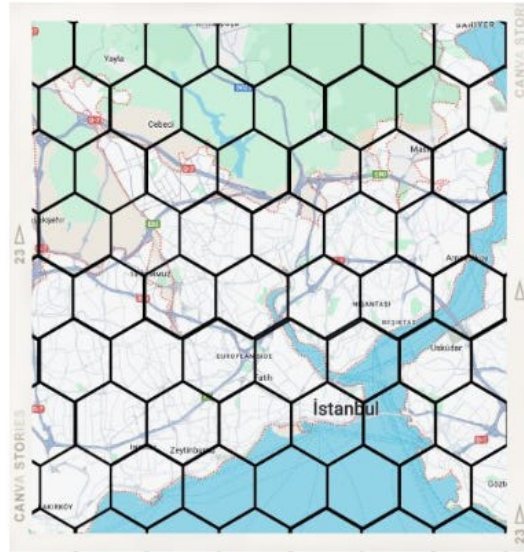
They need the index of the smallest possible area to find the nearby drivers accurately. Put another way, a high data resolution is necessary.

A small cell provides high data resolution. Yet storage costs increase if there are many smaller-sized cells.

While bigger-sized cells hide details in data.



smaller grid cells



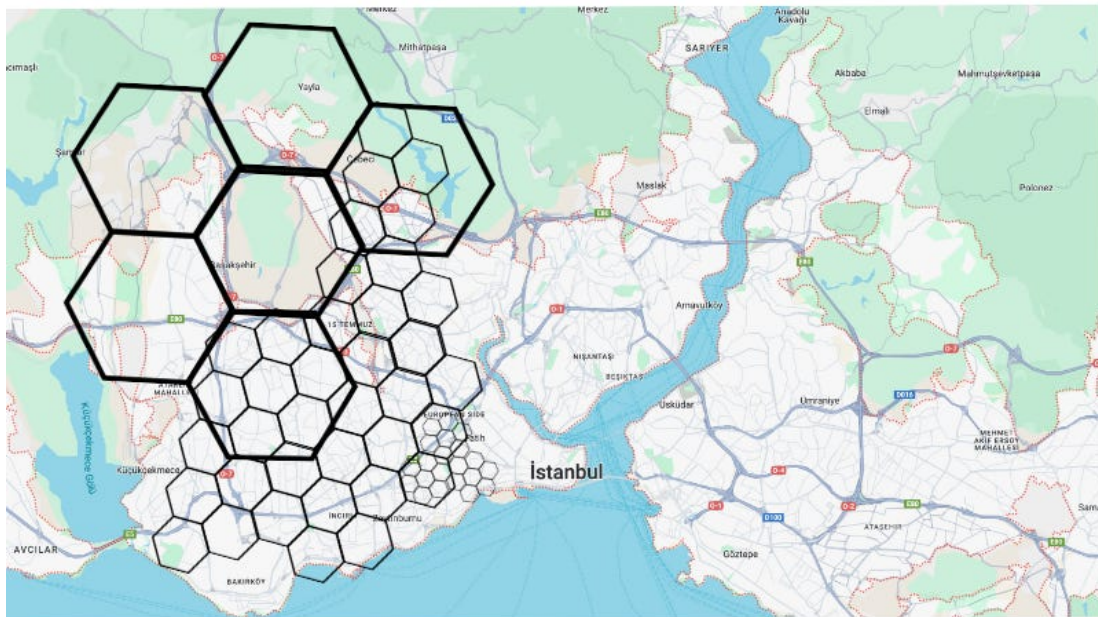
larger grid cells

newsletter.systemdesign.one

Data Resolution Based on Grid Size

So they created a hierarchical grid. And defined the smaller hexagonal cells as subdivisions of bigger hexagonal cells.

A hierarchical grid allows data compression. Because the areas with lower details are represented with fewer cells. While areas with higher details are represented by many cells.



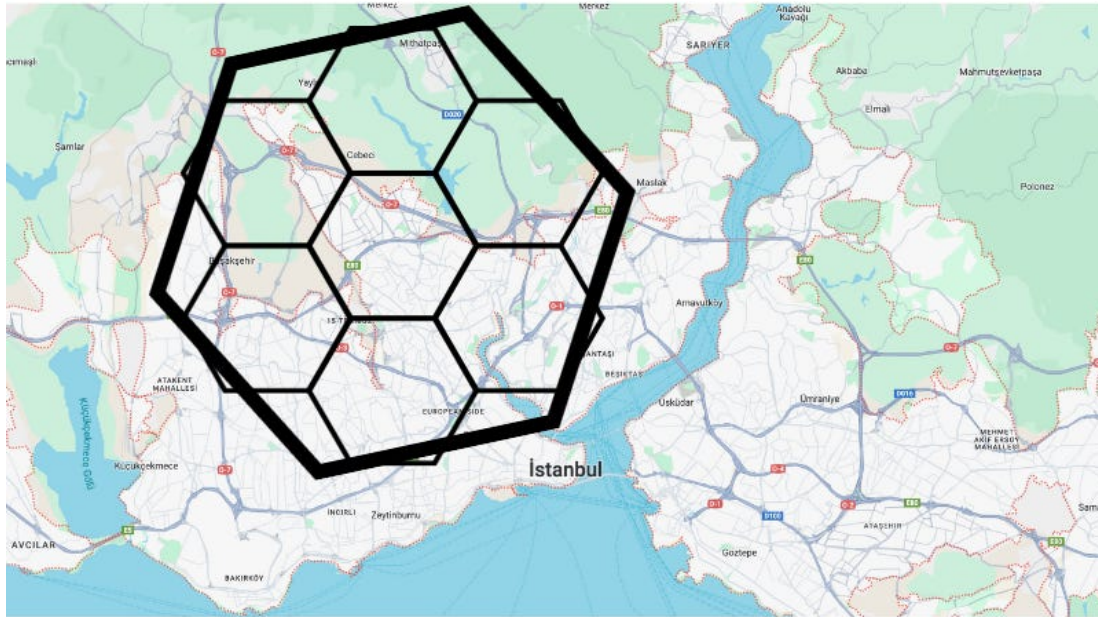
newsletter.systemdesign.one

Fewer Cells Representing Large Areas Without Details in
Hierarchical Grid

Also changing data resolution became easier with a hierarchical grid. Because the identifier of child cells could be truncated to find ancestor cells.

They created a global grid on H3 with 122 base cells. Yet 12 out of 122 base cells are *pentagons* because it's not possible to tile a sphere (Earth) only with hexagons. H3 treats the pentagon as a hexagon without a part.

Besides hexagons don't subdivide perfectly. So they subdivide a hexagon into 7 hexagons with a known amount of error. And the child hexagons get rotated to fit the shape of the parent hexagon.



newsletter.systemdesign.one

H3 Subdividing Areas Into Smaller Hexagons

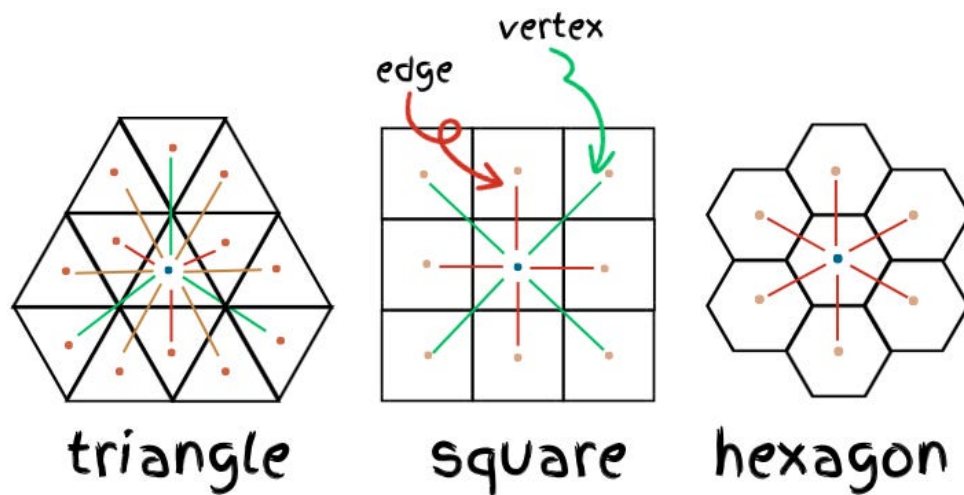
The H3 library supports 16 data resolution types. And can index an area as small as 1 square meter.

They use the cell identifier as the *sharding key* to partition the H3.

ii) Hexagonal Grid

They need distance between cells to find nearby drivers.

It's possible to create a grid with triangle, square, or hexagon-shaped cells.



newsletter.systemdesign.one

Distance Measurement in Grid

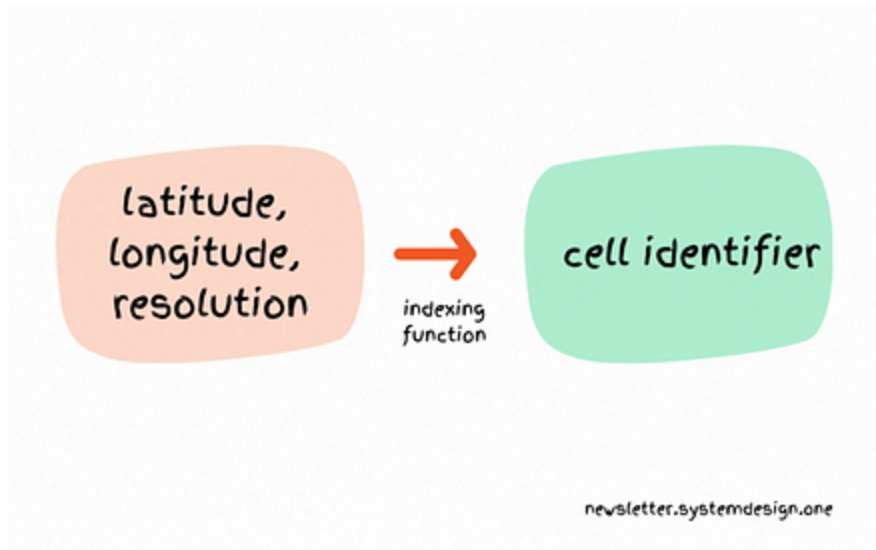
But H3 uses a hexagon as the cell shape to make it easier to measure the distance between 2 cells. Because each neighboring cell is the same distance away from a cell's center. It *simplifies distance measurement*.

While triangles have 3 distanced neighbors and squares have 2 distanced neighbors. Because some neighbors share an edge and others share a vertex.

So distance measurement between cells needs extra coefficients in triangles and squares.

iii) Hierarchical Algorithms

H3 supports the *indexing function* to find a cell in the grid efficiently. It accepts latitude, longitude, and resolution to return the cell identifier.



Finding a Cell Identifier Using Indexing Function

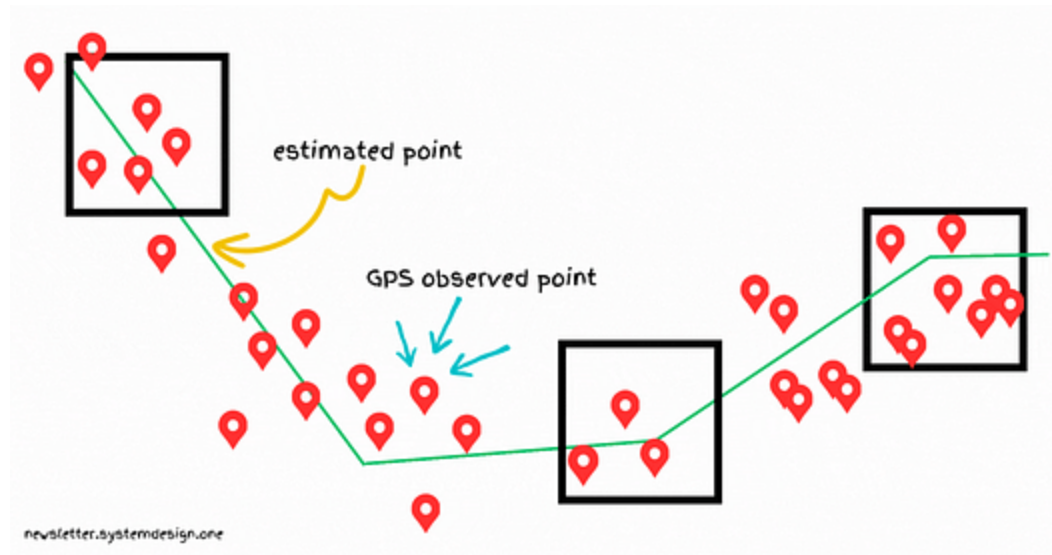
H3 finds the neighboring cells around a specific cell using the *kRing function*.

Besides H3 does *constant-time bitwise operations* to truncate the cell index. It makes switching between data resolutions easy.

2. Location Storage

Each driver app sends its location to the server every few seconds. But the GPS signals can get noisy and sparse due to a poor network.

The exact location is necessary to find nearby drivers. So they do *map matching*.



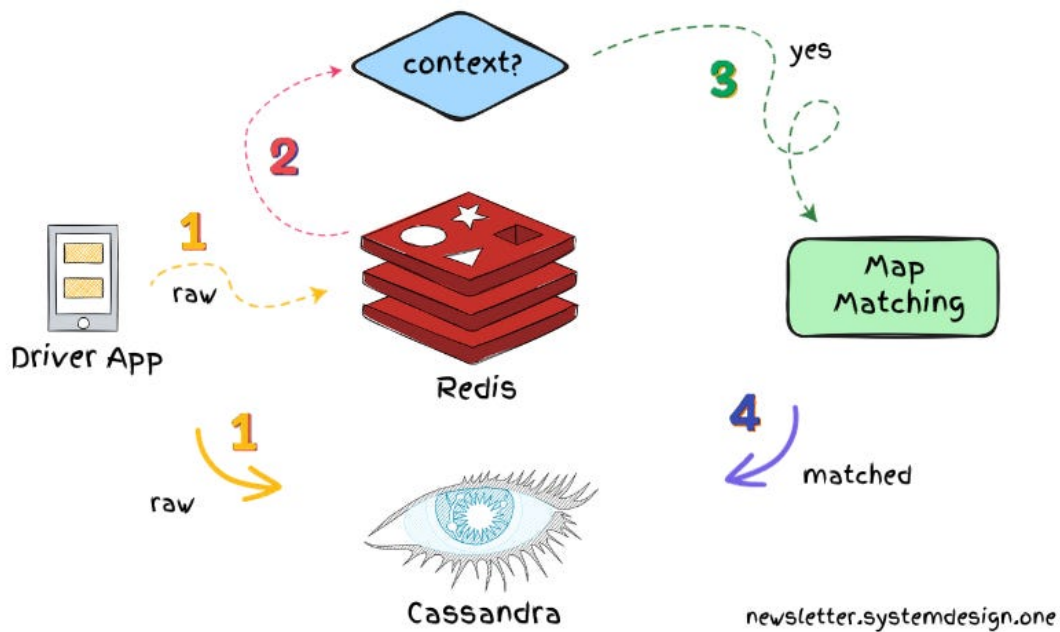
Map Matching

Map matching transforms raw GPS signals into actual road segments.

They use Apache Cassandra to store raw locations for long-term durability.

Apache Cassandra is a distributed database. Yet Cassandra is optimized for write operations.

So they add a Redis cache layer on top of it to shed the *read operations*.



Redis Buffering Data Points for Map Matching

Redis stores the recent locations of each driver. Also it buffers enough data points to do map matching.

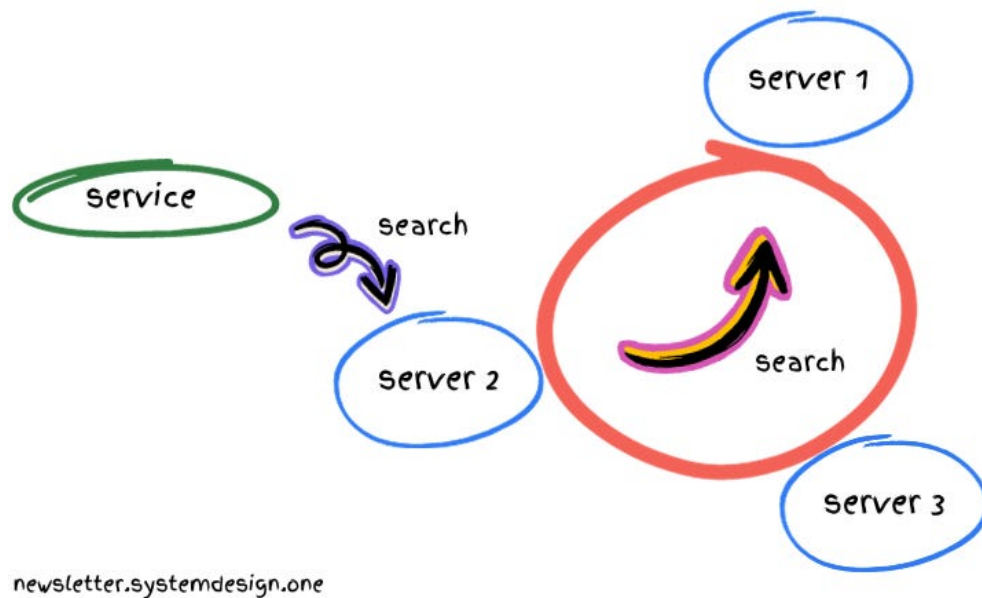
The map-matched data then gets stored in a separate schema on Cassandra.

3. Scalability

They use *ringpop* to partition the services for scalability.

Ringpop is Uber's consistent hash ring module that uses gossip protocol. It gets embedded in application services.

They use gossip protocol to track the server membership list and server health status.



Ringpop: Uber's Consistent Hash Ring for Distributed Coordination

A consistent hash ring lookup forwards a request to the responsible server.

They scale writes by adding more servers. While reads are scaled by adding replicas.

They use Thrift over Remote Procedure Calls (**RPC**) for efficient communication.

Besides every service is kept idempotent for high availability through retries.

They run a backup data center and route the traffic to it if the active data center fails.

It's necessary to find nearby drivers accurately for the best rider experience.

The current approach allowed them to scale to 1 million requests per second.

Consider subscribing to get simplified case studies delivered straight to your inbox:

✓ Subscribed



Follow me on [LinkedIn](#) | [YouTube](#) | [Threads](#) | [Twitter](#) | [Instagram](#)

Thank you for supporting this newsletter. Consider sharing this post with your friends and get rewards. Y'all are the best.

1 Referral



Leetcode
Master
Template

2 Referrals



Interview
Mistakes
to Avoid PDF

3 Referrals



Popular
Interview
Questions PDF

Share

UBER

NEARBY
DRIVERS





How Uber Computes ETA at Half a Million Requests per Second

NK · DECEMBER 3, 2023

[Read full story](#)



How PayPal Was Able to Support a Billion Transactions per Day With Only 8 Virtual Machines

NK · DECEMBER 26, 2023

[Read full story](#)

References

- [Location Serving & Storage in the Uber Marketplace](#)
- [Life of an Uber trip](#)
- [Scaling Uber's Real-time Market Platform](#)
- [H3: Uber's Hexagonal Hierarchical Spatial Index](#)
- [H3: Hexagonal hierarchical geospatial indexing system](#)
- [H3: Tiling the Earth with Hexagons](#)
- [Hierarchical Hexagons in Depth](#)